



Laboratório de Auditoria de Acesso

Validação de Privilégios RBAC e Prova de Eficácia da Segurança



Como Provar que a Política de Segurança Funciona de Verdade?

Abertura: Por que Auditar a Segurança?



Como sabemos que nossa política de segurança funciona de verdade e não apenas no papel?

Conceito de Auditoria de Segurança

Auditoria é Prova de Eficácia

O profissional de dados deve ser capaz não apenas de **criar** a segurança (GRANT), mas de **provar** que ela é eficaz (Auditoria). É um requisito essencial para conformidade (LGPD, HIPAA).

O Papel do Auditor

Nesta aula, cada aluno atuará como auditor, testando a política de segurança RBAC implementada na aula anterior. O foco é a **validação independente**.

Revisão: Teste de Hipóteses (PoLP)

Toda Política Tem Duas Hipóteses

Para cada usuário/papel, há uma hipótese de permissão (o que deve funcionar) e uma hipótese de negação (o que deve falhar).

Hipótese de Sucesso

O comando deve ser executado (Ex: Leitor faz SELECT). O resultado esperado é **SUCESSO**.

Hipótese de Negação

O comando deve ser bloqueado (Ex: Leitor faz UPDATE). O resultado esperado é **ERRO** (Acesso Negado).

Bloco 1: A Tabela de Validação (Relatório de Auditoria)

 Instrução: Crie sua Tabela de Validação (Obrigatória)

Cada aluno deve criar uma tabela simples (em um documento de texto ou planilha) para registrar os resultados de seus testes. Esta tabela atua como o ****relatório de auditoria****.

Usuário / Papel	Comando de Teste	Resultado Esperado	Resultado Real	Status (Conforme/Falha)
João (Leitor)	SELECT * FROM Vendas;	SUCESSO	SUCESSO	Conforme
João (Leitor)	UPDATE Produtos SET...	ERRO (Acesso Negado)	ERRO	Conforme
Maria (Operador)	DELETE FROM Vendas...	ERRO (Acesso Negado)		
Maria (Operador)	INSERT INTO Vendas...	SUCESSO		
Pedro (Admin)	DROP TABLE Produtos;	SUCESSO		

Bloco 2: Testes de Leitura (SELECT) - Hipótese de Sucesso

 ****Instrução****: Logue separadamente com cada usuário e execute os comandos abaixo. O resultado esperado é ****SUCESSO**** (Conforme o PoLP).

Teste 1: Leitor (João)

Papel: leitor_relatorios

```
SELECT * FROM Vendas;
```

 Resultado Esperado: SUCESSO

Teste 2: Leitor (João)

Papel: leitor_relatorios

```
SELECT * FROM Produtos;
```

 Resultado Esperado: SUCESSO

Teste 3: Operador (Maria)

Papel: operador_vendas

```
SELECT * FROM Clientes;
```

 Resultado Esperado: SUCESSO

Bloco 2: Testes de Escrita/Alteração (UPDATE e INSERT) - Hipótese de Negação



Objetivo: Validar se a restrição de privilégios (UPDATE, INSERT) está funcionando. O resultado esperado é o ****ERRO de permissão****.

🔪 Teste 4: Tentar Alterar Preço (UPDATE)

Usuário: João (leitor_relatorios)

```
UPDATE Produtos SET preco = 99 WHERE id = 1;
```

✖ **Resultado Esperado: ERRO (Acesso Negado)**

****Registro**:** Anote a mensagem de erro. Marque ****"Conforme"**** se o erro ocorreu (pois o erro é o resultado esperado).

+ Teste 5: Tentar Inserir Venda (INSERT)

Usuário: João (leitor_relatorios)

```
INSERT INTO Vendas (colunas) VALUES (valores);
```

✖ **Resultado Esperado: ERRO (Acesso Negado)**

****Registro**:** Anote a mensagem de erro. Marque ****"Conforme"**** se o erro ocorreu (pois o erro é o resultado esperado).

Bloco 2: Testes de Exclusão (DELETE) e DDL - Risco Crítico

Usuário: Maria (operador_vendas) - Foco em validar a negação de privilégios perigosos

6 Teste: Tentar Excluir Produto (DELETE)

Objetivo: Validar se o Operador não pode remover dados críticos (DELETE).

```
DELETE FROM Produtos WHERE id = 1;
```

✖ **ERRO - Acesso Negado (Operador não tem DELETE)**

7 Teste: Tentar Comando DDL (DROP)

Objetivo: Validar se o Operador não pode alterar a estrutura do banco (DDL).

```
DROP TABLE Vendas;
```

✖ **ERRO - Acesso Negado (Operador não tem DROP)**

⚠ Risco Crítico: DELETE e DDL

O DELETE e os comandos DDL (DROP, ALTER) representam o maior risco para a integridade e disponibilidade do banco de dados. A negação correta desses privilégios é a prova de que o PoLP está funcionando.

🚫 Erro de Permissão vs Erro de Sintaxe

A negação correta deve ser uma mensagem específica sobre a **falta de privilégio** ('Access denied for user...'). Se o erro for de sintaxe, o teste é inválido. O erro de permissão é o **resultado esperado** para estes testes.

Bloco 3: Análise de Falhas de Validação e Conserto

1. Análise de Falhas ("Status: Falha")

! O que significa uma Falha?

Se um teste retornou **"Falha"** (Ex: O Leitor conseguiu rodar o UPDATE), significa que a política de segurança está **incorreta**. O resultado real não corresponde ao resultado esperado.

Exemplo de Falha Hipotética

🚩 Falha: Leitor Conseguiu UPDATE

Teste: Usuário João (Leitor) executa `UPDATE Produtos SET preco = 99...`

Esperado: ERRO (Acesso Negado)

Real: SUCESSO

Status: FALHA

2. Tarefa de Conserto: O Comando REVOKE

🔧 Corrigindo a Falha

O aluno deve assumir o papel de **Administrador** e escrever o comando **REVOKE** necessário para corrigir a falha, removendo o privilégio concedido indevidamente.

Comando REVOKE (DCL)

```
-- Conserto para a Falha Hipotética
REVOKE UPDATE ON nome_do_banco.Produtos
FROM 'leitor_relatorios'@'%';
```

🔄 Revalidação

Após o `REVOKE`, o aluno deve **re-executar o teste** para confirmar que o status agora é **CONFORME** (o resultado real deve ser ERRO).

Exemplo Prático de Conserto: Comando REVOKE

1. Falha Hipotética (Status: Falha)

! O Leitor Conseguiu Alterar o Preço!

****Teste 4 (Leitor)**:** `UPDATE Produtos SET preco = 99 WHERE id = 1;`

****Resultado Real**:** SUCESSO (O resultado esperado era ERRO).

****Conclusão**:** A política de segurança está incorreta. O papel `leitor\_relatorios` tem a permissão `UPDATE` indevidamente.

2. O Conserto: Revogando o Privilégio Indevido

Assumindo o papel de Administrador, execute o comando ****REVOKE**** para remover a permissão de `UPDATE` do papel `leitor\_relatorios`.

```
-- Comando REVOKE para corrigir a falha  
REVOKE UPDATE ON nome_do_banco.Produtos FROM 'leitor_relatorios'@'%';
```

3. Revalidação: Prova de que o Conserto Funcionou

↺ Re-execute o Teste 4

Após o `REVOKE`, o aluno deve re-executar o Teste 4. O resultado esperado agora é ****ERRO (Acesso Negado)****. Se o resultado for ERRO, o status na Tabela de Validação muda para ****CONFORME****.

Apresentação de Resultados e Problemas Encontrados



****Instrução**:** Cada aluno deve compartilhar com a turma um dos testes que executou e o comando ****REVOKE**** que utilizou para corrigir uma falha.



1. Compartilhe um Teste

Qual comando SQL você executou e qual foi o resultado real? (Ex: Leitor fez SELECT, SUCESSO).



2. Foco na Falha

Houve alguma falha (Resultado Real $\neq$ Resultado Esperado)? Qual foi o problema na política de segurança?



3. O Conserto (REVOKE)

Qual comando ****REVOKE**** você utilizou para corrigir a falha e revalidar o teste? (Aplicação prática do DCL).

Bloco 4: Consolidação e Responsabilidade Profissional



A segurança de dados é um trabalho de ****disciplina contínua****. O script de GRANT é apenas o início.

Script de GRANT

É o ****Início**** da Política



1. Disciplina e Consistência

A política de segurança deve ser aplicada de forma uniforme. Qualquer exceção deve ser documentada e testada.



2. Teste Constante (Auditoria)

Sempre que houver uma mudança de papel, usuário ou tabela, a política deve ser revalidada com o roteiro de testes.

Auditoria e Teste Constante

É a ****Garantia**** da Política



3. Prova de Eficácia

O relatório de auditoria (Tabela de Validação) é a prova de que a segurança está sendo mantida, essencial para auditorias.

Encerramento: Documentação de Segurança e Próximos Passos



A Tabela de Validação preenchida atua como ****PROVA (EVIDÊNCIA)**** de que as medidas de segurança foram testadas e aprovadas. Essencial para auditorias internas e externas.



Próximo Passo: O Fluxo de Dados (Pipeline)

A segurança é apenas uma parte da governança. Na próxima aula, vamos analisar o ****fluxo de dados**** e sua ****transformação**** (Pipeline).

****Preparação****: Revisar os conceitos de ETL/ELT e as camadas Bronze, Prata e Ouro do Data Lake.